

reframe UI 기획서

reframe UI 기획서 — 크롭 편집 콘솔 + 웹 URL 송출

작성일: 2026-07-03 · 상태: 기획 · 상위 문서: [PLAN.md](#)

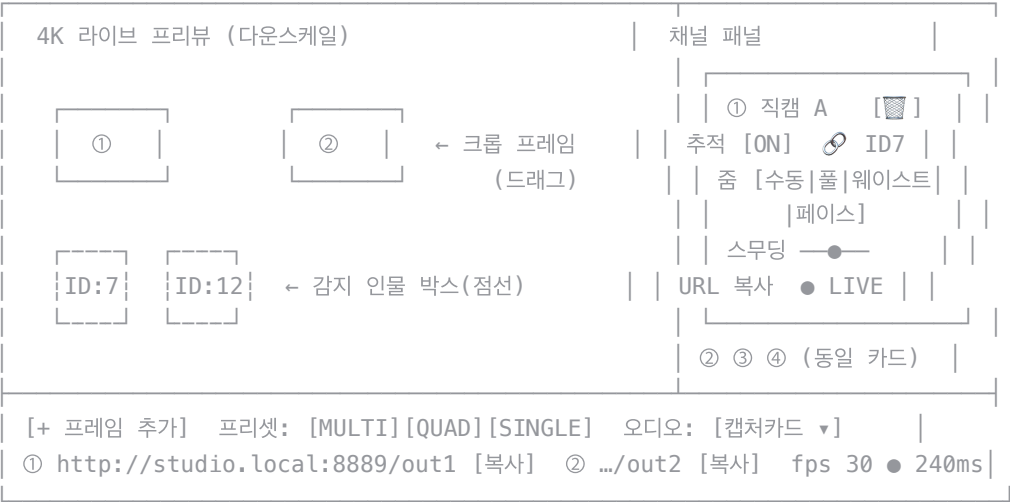
1. 요구사항

- 1. 하나의 UI에서 4K 영상 위의 크롭 프레임을 이동·축소(확대)·삭제·추가하며 리프레임
- 2. 프레임별 **트래킹 활성화/비활성화** — 활성화 시 감지된 인물을 따라다님
- 3. 각 프레임(출력 채널)을 **웹 URL로 변환**해 OBS에 소스로 올릴 수 있어야 함
- 4. **영상·오디오 딜레이 최소** (A/V 동기 유지 포함)

출력이 웹 URL이어야 하므로 컨트롤 UI 자체도 웹앱으로 만든다 (설치·크로스머신 접근 문제 동시 해결).

2. 화면 구성

단일 페이지. 좌측 프리뷰 캔버스 + 우측 채널 패널 + 하단 출력 바.



프리뷰 캔버스 레이어 구조

- **하단**: 4K 입력의 다운스케일 라이브 영상 (WHEP preview 채널, 1280×720)
- **상단**: HTML 캔버스 오버레이 — 감지 인물 박스(점선+트랙ID), 크롭 프레임(채널 색상 실선+번호)
- 오버레이 데이터는 WebSocket으로 ~10Hz 수신해 클라이언트에서 그림 → 영상 인코딩에 오버레이가 섞이지 않고, 출력 채널은 항상 클린 피드

3. 인터랙션 명세

크롭 프레임 편집

조작	동작
프레임 내부 드래그	이동 (프레임 경계 클램프)
모서리 핸들 드래그	크기 조절 — 16:9 고정. 작게 = 줌인, 크게 = 와이드
프레임 선택 + Delete키 / 카드의 	삭제 (해당 URL 즉시 오프라인)
+ 프레임 추가	새 채널 생성, 중앙에 기본 크기로 배치 (최대 4)
프리셋 버튼	MULTI/QUAD/SINGLE 구성으로 4개 프레임 일괄 재배포 (기존 PLAN.md의 3모드가 프리셋으로 흡수됨)

- 크롭 크기 하한: 480×270 (4배 업스케일 초과 방지 경고 표시)
- 모든 편집은 라이브 중 즉시 반영 (출력 스트림 끊김 없음 — 크롭 좌표만 변경)

트래킹

상태	동작
추적 OFF (기본)	고정 크롭 — 수동 배치 그대로
추적 ON + 대상 미 지정	프리뷰의 인물 박스가 클릭 가능 상태로 하이라이트 → 클릭한 인물에 바인딩 (🔗 ID 표시)
추적 중	크롭 중심이 One Euro 스무딩으로 인물을 따라감. 크기는 수동 크기 유지, 또는 줌 프리셋(풀/웨이스트/페이스) 선택 시 인물 크기 비례
인물 소실	마지막 위치 홀드 + 카드에 "대상 소실" 배치. N초 후에도 미복귀 시 유지(끊지 않음), 재감지 시 재바인딩
스무딩 슬라이더	One Euro min_cutoff 매핑 — 왼쪽 = 부드럽게(랙↑), 오른쪽 = 기민하게(지터↑)

상태 표시

- 채널 카드: ● LIVE(초록) / 대상 소실(노랑) / 오프라인(회색)
- 하단 바: 파이프라인 fps, 엔드투엔드 지연 추정치(ms), 시청자(OBS 연결) 수

4. 송출 아키텍처 — 내부 신호 우선, 웹 URL은 필요할 때만

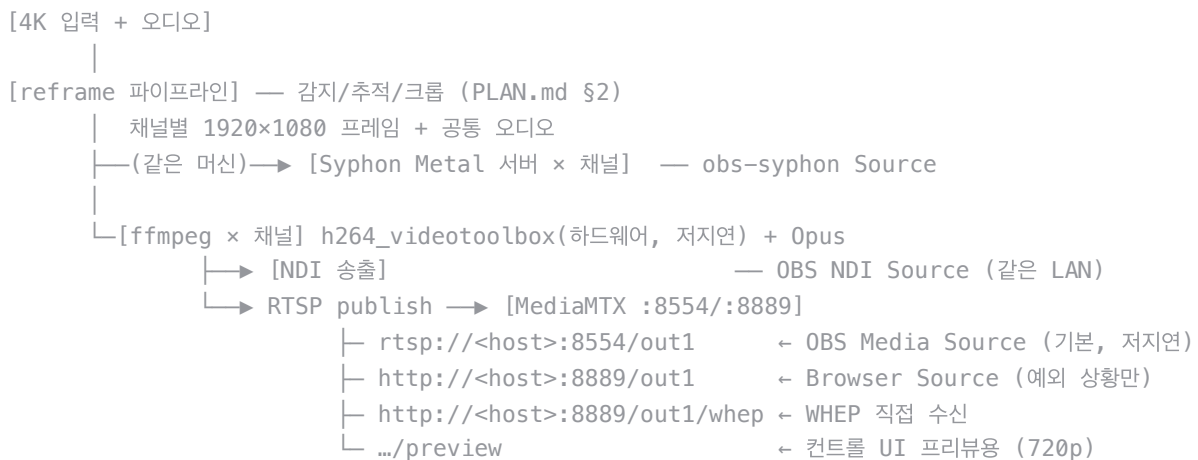
수정 이력(2026-07-03): 최초안은 WebRTC를 기본으로 잡았으나, 이 신호는 인터넷을 나가지 않는 내부(같은 머신 또는 같은 LAN) 신호이므로 재검토함. WebRTC의 핵심 가치(ICE/NAT 통과, 손실 적응형 지터버퍼, 혼잡 제어)는 전부 인터넷 회선의 불안정성에 대응하는 장치다. 내부 신호에는 그 문제가 없는데 지연 비용(지터버퍼 100-200ms)만 그대로 부담하는 구조였다. → OBS와의 관계에 따라 프로토콜을 분기하고, WebRTC(Browser Source)는 "진짜 일반 브라우저로 봐야 하는" 경우로 한정.

선정 매트릭스

상황	방식	지연	OBS 연동
파이프라인과 OBS가 같은 머신	Syphon (GPU 텍스처 공유)	~0 (인코딩 생략)	obs-syphon 플러그인의 Source (URL 아님)
같은 LAN, 다른 머신	NDI	~1프레임(16-33ms)	네이티브 NDI Source 플러그인 (DistroAV/obs-ndi)
URL 문자열은 필요하지만 브라우저일 필요는 없음	RTSP (MediaMTX가 같은 입력을 무료로 동시 서빙)	WebRTC보다 낮음 (지터버퍼 부담 적음)	OBS Media Source 에 <code>rtsp://host:8554/out1</code> 입력
진짜 일반 웹브라우저로 봐야 함 (모니터링용 태블릿/폰, NDI 런타임 없는 PC)	WebRTC/WHEP (MediaMTX)	300-500ms	Browser Source

기본 송출은 Syphon(동일 머신) 또는 NDI(동일 LAN)로 하고, "채널을 웹 URL로 변환" 요구는 **RTSP URL을 OBS Media Source에 넣는 경로**로 충족한다 — URL을 복사해 붙여넣는 사용성은 유지하면서 WebRTC의 지터버퍼 지연은 피한다. WebRTC/Browser Source는 OBS 없이 순수 브라우저로 모니터링해야 하는 예외 상황에만 쓴다.

아키텍처



- UI의 URL 복사 버튼은 기본적으로 `rtsp://` 주소를 복사하고, 채널 카드에 "브라우저용 URL 보기" 토글을 뒤서 WebRTC 주소도 필요 시 꺼내 쓸 수 있게 한다.
- MediaMTX는 하나의 ffmpeg publish를 RTSP/WebRTC/HLS로 동시에 서빙하므로 별도 인프라 없이 두 경로를 공짜로 확보한다.

지연 예산

구간	RTSP(기본)	WebRTC(예외)
캡처 (4K30, 1프레임)	33ms	33ms
감지+추적+크롭	15-25ms	15-25ms
h264 VideoToolbox 인코딩 (저지연 프리셋, B 프레임 0)	5-15ms	5-15ms
MediaMTX 릴레이	<5ms	<5ms
수신측 디코드+버퍼	~30-80ms (OBS Media Source, 인터넷 손실 대비 버퍼 불필요)	100-200ms (WebRTC 지터 버퍼)
합계	~100-150ms	~250-350ms

- **오디오 동기:** 캡처카드 오디오를 ffmpeg에서 영상과 같은 타임스탬프로 먹싱(Opus 20ms 프레임) → RTP 타임스탬프로 A/V 동기가 프로토콜 수준에서 유지됨. 오디오를 별도 경로(예: OBS 직접 캡처)로 받으면 영상 처리 지연만큼 어긋나므로, **오디오는 반드시 스트림에 실어 보낸다** (UI에서 소스 선택).
- **폴백:** 오디오가 불필요한 모니터링 채널은 MJPEG 엔드포인트(/out1.mjpg)도 가능 — 지연 최소(~1프레임)지만 무음.

5. 기술 스택 및 API

계층	선정	비고
백엔드	Python FastAPI + 기존 reframe 파이프라인	컨트롤 REST + WebSocket
프론트	단일 HTML + vanilla JS 캔버스	빌드 도구 없이 시작, 복잡해지면 React 승격
스트리밍	MediaMTX (단일 바이너리) + ffmpeg 서브프로세스	brew install mediamtx ffmpeg
상태 동기	WebSocket: 서버→클라 (인물박스, 크롭 좌표, fps) / 클라→서버 (편집 커맨드)	~10Hz

API 스케치

GET /api/state	전체 상태 (채널, 트랙, URL)
POST /api/channels	채널 추가 {x,y,w,h}
PATCH /api/channels/{id}	이동/리사이즈/설정 {x?,y?,w?,h?,tracking?,target_track?,zoom?,smoothing?}
DELETE /api/channels/{id}	채널 삭제
POST /api/preset/{multi quad single}	프리셋 적용
WS /ws	오버레이 데이터 + 편집 커맨드

6. 구현 단계 (PLAN.md Phase 갱신)

- Phase 3 (기존 "프로덕션 출력")을 본 문서로 대체:
 1. MediaMTX + ffmpeg 1채널 송출 PoC — OBS 브라우저 소스에서 지연 실측
 2. 4채널 + 오디오 믹싱, A/V 동기 검증
 3. FastAPI + 프리뷰 스트림 + 캔버스 오버레이 (읽기 전용 콘솔)
 4. 크롭 편집 인터랙션 (이동/리사이즈/삭제/추가/프리셋)
 5. 트래킹 바인딩 UI (인물 클릭 → 채널 연결) + 소실 처리
- 리스크 추가: OBS 브라우저 소스의 WebRTC 코덱 호환(h264 baseline 권장), 다채널 동시 인코딩 부하 (VideoToolbox 하드웨어 세션 4개 — M시리즈에서 여유 있으나 실측 필요)

7. 참고 자료

- [MediaMTX · WebRTC 읽기 문서](#) — WHEP URL 형식, 지연 튜닝
- [OBS + WebRTC 초저지연 구성 \(2026-03\)](#)
- [MediaMTX 브라우저 소스 실사용 사례 \(300-500ms\)](#)
- 인코더 주의: x264 사용 시 preset Fast 이하 + tune Low Latency 필수 사례 → VideoToolbox 저지연 설정으로 회피